Failure Containment and Executable Actions in Language-Based Multi-UAV Planning

Anonymous Author(s)
Affiliation
Address
email

Abstract: Language-based robot planners can produce fluent plans despite missing evidence or conflicting resources. We compare four validation-pipeline configurations on 1,420 controlled derivatives of 284 held-out MultiUAV-Plat tasks using frozen Qwen2.5-3B and 7B checkpoints. Stage-wise validation eliminated unsupported continuation on all 568 non-executable cases, but failed every one of 852 executable cases. A zero-call policy that always blocks achieved 20.0% strict success, exceeding the 7B stage-wise configuration's 16.1%, showing that containment alone does not establish useful discrimination. A post-hoc audit also found that the frozen validator rejected 43 of 249 fully instantiated upstream reference plans. A paired replay that held prompts, generations, validators, and call count fixed found no placement-only change in containment, false non-execution, or static plan fidelity. The study is therefore a negative result for the frozen pipelines: changing enforcement order alone did not recover executable planning. A separate repaired, observation-grounded closed-loop controller completed six of 15 training-split development missions under the upstream official task checker. Three missions ended with false checks, five exhausted GPU memory, and one was interrupted. An additional compact-context rerun recovered one of the memory-failed missions with 283 accepted commands and an official true check on a different GPU. The original failures remain reported. These inspected development cases show that bounded model-selected actions can execute, not that validation placement improves held-out competence or physical safety.

Keywords: physical AI safety, robot planning, failure containment, evaluation

1 Introduction

An open-ended language interface can hide a consequential physical-AI failure: a response may appear executable while relying on evidence that is absent from the robot's visible context. Prior systems combine language models with supervision, structured planning, and motion constraints [1, 2, 3, 4]. These systems motivate modular safety checks, but they do not resolve how to evaluate a planner that avoids unsafe continuation by refusing useful requests.

We study this distinction using controlled MultiUAV-Plat derivatives [5]. Here, *containment* means that a pipeline does not advance on a registered non-executable case. *Static plan fidelity* means conformance with frozen schema, endpoint, parameter, and command validators; it does not mean simulator or physical success. Our contributions are: (1) a paired five-variant benchmark for evidence availability and resource conflict; (2) a comparison of four complete validation pipelines with frozen model and prompt versions; and (3) a refusal-aware analysis that separates containment from executable competence, audits the validator against upstream plans, and replays identical traces under early versus deferred enforcement. We additionally report a separate development repair using bounded action options and observed geometry, retaining every mission and infrastructure failure. This repair is not a fifth configuration in the frozen comparison.

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

2 Experimental Design

Cases. The pinned source contains 1,500 tasks in 75 sessions. Deterministic duplicate and leakage checks retained 1,473 source tasks, split by session into 899 train, 290 calibration, and 284 test clusters. Each held-out cluster yields five linked cases: canonical execute, official-alias execute, missing-evidence clarify, restored-evidence execute, and resource-conflict block. The test matrix contains 1,420 cases per method and model: 852 executable and 568 non-executable. A stratified 30-cluster pilot checked construction quality; it was not a complete human audit.

Configurations. M1 is a one-call monolithic planner. M2 adds a deterministic post-plan gate. M3 builds and checks an evidence ledger before planning and validates provenance afterward. M4 is a two-call post-plan configuration that is *model-call-count-matched* to M3. The comparison does not isolate placement: M3 and M4 also differ in prompts, validation behavior, generated content, and token counts. We therefore compare complete pipeline configurations rather than claim a causal placement effect.

Models and outcomes. We use immutable Qwen2.5-3B-Instruct and Qwen2.5-7B-Instruct revisions with deterministic decoding. Registered outcomes are unsupported continuation on the 568 non-executable cases and strict success over all 1,420 cases. Executable success requires the correct disposition and a non-empty plan passing every static validator; non-executable success requires the exact clarify or block disposition without a prohibited plan. Registered paired differences are aggregated by source task and use 10,000 fixed-seed percentile-bootstrap draws.

Post-hoc reviewer diagnostics. We score a deterministic always-BLOCK reference over unique cases, repeat paired contrasts by resampling the 15 held-out sessions, partition every executable failure into one observed location, and pass fully instantiated upstream reference API plans through the frozen validator. We also replay each retained two-call M3 trace under early and deferred enforcement while holding the prompts, both generations, validators, and call count fixed. Unresolved upstream templates are excluded rather than imputed. These diagnostics do not change the registered analysis and require no new model calls.

3 Results

Figure 1 separates two positively oriented outcomes. For 3B, containment rates for M1–M4 were 21.0%, 99.6%, 100%, and 99.8%; all strict-success rates were zero. For 7B, containment was 31.7%, 97.7%, 100%, and 97.4%, while strict success was 0%, 0%, 16.1%, and 0.8%. Every nonzero strict success was a correct non-executable outcome. All eight model–method conditions had 0/852 executable static plan fidelity.

Always-BLOCK contained every non-executable case and correctly handled the 284 conflict cases, producing 284/1,420 (20.0%) strict success. It exceeded 7B M3's 228/1,420 (16.1%) while falsely blocking all 852 executable and all 284 clarification cases. Thus, perfect containment and the aggregate strict-success metric do not demonstrate useful discrimination.

The registered 7B M3 minus M1 differences were -0.6831 for unsupported continuation (95% interval $[-0.7324, -0.6338]$) and 0.1606 for strict success ($[0.1507, 0.1697]$). Session-clustered sensitivity preserved the directions: -0.6829 ($[-0.7220, -0.6373]$) and 0.1605 ($[0.1513, 0.1688]$), respectively. Only 15 sessions were available, so this is a robustness check rather than a rich hierarchical analysis.

Placement-only replay. Across both checkpoints, early and deferred enforcement produced identical unsupported continuation (0/568), false non-execution (852/852), and executable static plan fidelity (0/852). For 3B, both policies also had zero strict success. For 7B, deferred enforcement preserved 228 correct final model refusals or clarifications (16.1%), whereas the early-enforcement policy recorded none as exact decisions. Under the frozen scoring contract, deterministic-gate

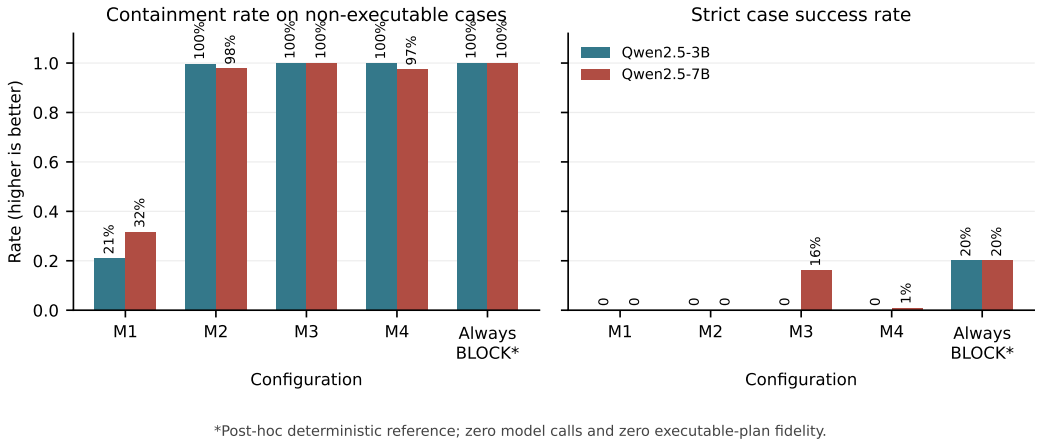


Figure 1: Containment and strict success, both higher-is-better. Always- BLOCK is a post-hoc zero-call reference with zero executable-plan fidelity.

Table 1: Paired M3 trace replay. Unsupported continuation is over 568 non-executable cases; false non-execution and static plan fidelity are over 852 executable cases; strict success is over all 1,420 cases.

Model	Enforcement	Unsupported	False nonexec.	Static fidelity	Strict success
3B	Early	0	852	0	0
3B	Deferred	0	852	0	0
7B	Early	0	852	0	0
7B	Deferred	0	852	0	228

outcomes record containment locations rather than exact CLARIFY or BLOCK decisions. The early-minus-deferred difference was -0.1606 (95% source-task-clustered interval $[-0.1697, -0.1507]$). Thus, enforcement timing alone altered neither unsupported continuation nor executable fidelity; M3's nonzero 7B success came from the second-call decision. Because both calls were retained, this replay makes no compute-saving claim.

Where M3 executable cases failed. For 3B M3, 544/852 executable cases stopped before planning and 308/852 failed output parsing. For 7B M3, 637/852 produced a model non-execution decision, 187/852 failed output parsing, and 28/852 stopped before planning. These mutually exclusive counts show complete failure on executable cases through preplanning stops, non-execution decisions, or parse errors rather than successful executable planning.

Validator audit. Of 284 upstream reference plans, 249 were fully instantiated and 35 contained unresolved or missing template fields. The frozen validator accepted 206/249 (82.7%) and rejected 43/249 (17.3%): one at endpoint schema, three at parameter grounding, and 39 at safety bounds. Validator mismatch therefore is a plausible contributor to the zero-fidelity result, although it does not explain every model failure.

Compute. A separate exploratory campaign measured 30 clusters, all five variants, four methods, two models, and three repetitions on one RTX 3090. Relative to M1, M3 increased 3B duration by 8.86–9.49 s and board energy by 1,565–1,681 J, but reduced 7B duration by 6.23–6.82 s and board energy by 1,153–1,245 J. Relative to model-call-count-matched M4, 7B M3 generated 446.82 fewer output tokens and used 5,669–6,279 J less board energy. These differences are confounded by termination behavior and generated length; they are not evidence that stage-wise validation is intrinsically cheaper.

Table 2: Original v11 outcomes for all 15 development missions, including two repair cases. Official checker completion is distinct from static fidelity; incomplete and memory- failed runs have no verified final mission outcome.

Outcome	Missions
Official task check true	6
Terminal official task check false	3
CUDA out-of-memory failure	5
Deadline interruption; incomplete	1
Total registered cohort	15

4 Observation-Grounded Development Repair

Action abstraction and execution boundary. The frozen experiment tests complete static plans. To investigate whether a revised interface can actually execute useful commands, we separately developed a closed-loop controller. Qwen2.5-3B selects one identifier from a deterministic menu of takeoff, observed-target movement, area-sweep, or canvas-discovery actions. The model cannot emit executable code or invent API coordinates. The menu uses AGENT-local observations, remembered observed targets and obstacles, public canvas dimensions, and prior actions. Collision-aware visibility-graph routing uses buffered observed obstacle geometry; each movement step is capped at 0.8 times the drone's perception radius. The controller refreshes observations and checks the upstream official task predicate after accepted commands, stopping immediately on a true result. The checker supplies a completion signal, not hidden coordinates or reference plans. These engineered bounds do not certify physical collision avoidance.

Development protocol. The cohort contains 15 canonical training tasks, one per scenario/difficulty stratum, from the existing reviewed construction pilot. Two previously inspected missions were used during repair and retained in reporting; the other 13 were registered before their sweep. The original v11 sweep uses one fixed controller commit, pinned 3B checkpoint, float16, greedy decoding, seed 17, 256 generated tokens per call, and a 512-command ceiling. No weights were trained. Unlike M1–M4, this controller can make hundreds of calls and receive execution feedback; therefore its outcomes are not a matched comparison. Formation and photography missions remain in the cohort although the menu does not implement those actions. Infrastructure corrections changed scheduling, not the controller. Its outcome counts remain fixed; the later v12 rerun is reported separately.

Original v11 results and remaining failures. Six missions completed with 47, 55, 193, 216, 241, and 258 accepted commands; each accepted command was selected through a model call. The three terminal false outcomes exhausted available bounded actions after 460, 487, and 389 accepted commands; the 487-command run also made one model STOP call. One of these tasks requires formation, which the menu cannot express. The other two show that extensive accepted movement does not establish sufficient task coverage. Five runs failed with CUDA out-of-memory, including two on GPUs with about 24 GB; their raw traces are retained. A sixth-hour job deadline interrupted another mission after 411 journaled commands. The recorded memory errors establish a resource limit, not its sole cause. Retained prompts grow with serialized visit history and geometry; a memory leak has not been demonstrated. Multi-hour runs also remain a practical limitation.

All terminal traces and command/event journals were copied and verified against their recorded SHA-256 values. Appendix A lists every case and the immutable execution bindings. Hardware was heterogeneous and resource costs were not controlled in this development sweep. No energy advantage, held-out success rate, or improvement of M1–M4 is inferred. The six completions are evidence of a working execution path, not a resolution of the main study's generalization or validation-placement question.

Single-case memory repair. In a separately registered v12 follow-up, the controller retained full observed geometry and visit history for deterministic routing but sent the model a compact action-selection view, with a 4,096-token admission limit and no silent truncation. Previously memory-failed task `05e12d8e` completed with 283 model calls, 283 accepted commands and an official true task check. Input length peaked at 1,333 tokens, compared with 10,762 completed input tokens before its v11 failure. Raw and journal hashes were verified. The new run used a 32 GB V100 rather than the prior approximately 24 GB GPU; hardware and prompt contract changed together. This single inspected-training recovery therefore does not isolate a causal memory effect, establish a held-out success rate, or replace the original 15-case outcomes.

5 Discussion and Limitations

The central result is containment without competence. M3 suppresses unsupported continuation, but an indiscriminate block policy is stronger on the aggregate success metric and no frozen configuration produces a valid executable plan. The upstream-reference-plan audit additionally shows that the evaluation mechanism itself rejects a nontrivial fraction of upstream plans. The controlled replay further shows that changing deterministic enforcement order alone did not alter unsupported continuation or executable fidelity. The separate repair shows that an observation-grounded action interface can complete some official development missions, while exposing coverage, capability, and memory limits. It changes the representation, feedback, and call budget together, so it cannot identify which change caused execution success. A stronger claim requires re-audited validators and prospective matched evaluation on independent missions.

The study uses controlled derivatives rather than live operator missions, two sizes from one model family, a 30-cluster human pilot, and one GPU for resource measurement. M1–M4 perform no official simulator or physical execution; the separate development controller executes against the upstream software environment and task checker, not physical UAVs. Its training-split missions were inspected, two guided repair, and no new independent mission data were available. The source-task bootstrap is registered; the session bootstrap is post-hoc and has only 15 units. The resource campaign has three repetitions, and board energy is not whole-system or UAV energy. These limits prevent claims of general robot safety, practical mission capability, or broad causal superiority of one validation placement. The placement replay is conditional on retained M3 traces and is not a prospective compute experiment.

6 Conclusion

Validation-pipeline configurations can change where failures are contained, but containment is not evidence of planning competence. In this controlled study, stage-wise validation eliminated unsupported continuation while failing every executable case, underperforming always-BLOCK on aggregate strict success, and interacting with a validator that rejected 17.3% of auditable upstream plans. In replay of identical retained traces, enforcement timing changed neither unsupported continuation nor executable fidelity. The defensible contribution is a refusal-aware evaluation protocol, a negative frozen-study result, and a separately documented development repair that achieves six official mission completions while retaining nine unsuccessful or unresolved outcomes. Bounded executable actions address the absence of any execution demonstration, but do not establish held-out reliability or the superiority of a validation placement. One additional compact-context rerun recovered a previously memory-failed development mission, without establishing generalization or a matched hardware advantage.

References

[1] A. Nazzari, R. Rubinacci, and M. Lovera. TACOS: Task agnostic coordinator of a multi-drone system. *Drones*, 10(4):251, 2026. doi:10.3390/drones10040251.

[2] M. Zhao, S. Jain, and S. Song. RoCo: Dialectic multi-robot collaboration with large language models. *arXiv preprint arXiv:2307.04738*, 2023.

[3] S. Nayak et al. LLaMAR: Long-horizon planning for multi-agent robots in partially observable environments. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[4] M. Majid and A. Y. Majid. CommandSwarm: Safety-aware natural language-to-behavior-tree generation for robotic swarms. *arXiv preprint arXiv:2605.07764*, 2026.

[5] S. Zhang, Q. Li, Y. Zang, X. Huang, Y. Fu, and C. Zhu. MultiUAV-Plat: An llm-oriented platform, benchmark and framework for multi-uav collaborative task planning. *arXiv preprint arXiv:2606.31073*, 2026.

A Development Outcomes and Reproduction Bindings

Table 3 retains all original v11 outcomes. "Memory" means a preserved CUDA out-of-memory exception, not an official false result. "Interrupted" means the job deadline terminated an incomplete run. Counts for these outcomes must not be omitted when discussing the six completions. The two repair cases preceded the registered remaining-case sweep. No inferential confidence interval or population success rate is reported for this inspected cohort.

Table 3: Original v11 development cohort. Calls are completed model generations; the interrupted run has only a partial 411-row command journal.

Task ID	Session ID	Attempt	Outcome	Calls
`4cea97cd`	`bad1e9e5`	a25 (repair)	True	193
`519784ca`	`0f23ff1e`	a26 (repair)	True	241
`4c8040f9`	`04cd6269`	a27	True	47
`1036bedf`	`3ec31656`	a28	Interrupted	–
`a45a0c87`	`cc05a425`	a29	Memory	5
`ac5f85f8`	`72ccacd0`	a29	Memory	118
`6c4a6010`	`0567e470`	a29	Memory	123
`cf6f176c`	`793e7745`	a30	False	460
`9b355a03`	`139a4109`	a30	False	488
`91130026`	`13b94391`	a30	True	55
`14fd1139`	`3973d0d6`	a30	True	216
`c96a0146`	`550aa487`	a30	False	389
`67cbff58`	`b3c8ded2`	a30	True	258
`05e12d8e`	`e76728cc`	a30	Memory	231
`1905e0ad`	`35ce64d8`	a30	Memory	202

The controller execution commit is `9ca784efa16faab5cc1c9b261c7c509be6509d11`; the `Qwen/Qwen2.5-3B-Instruct` revision is `aa8e72537993ba99e69dfaafa59ed015b17504d1`; the upstream commit is `1794e45e421fb5de03094f0b63f9ca95f86ab42f`. The benchmark SHA-256 is `b5040097d2bfdd44600f3bf486fdb43ee3eb1247fec9c67900b5fcda6feb94a3`. The runner is `scripts/run_multiuav_train_model_loop_v2.py`; the a25–a30 Kubernetes manifests under `infra/nautilus/` bind its exact arguments. Per-attempt summaries under `outputs/evaluations/multiuav_train_model_loop_v2/` bind raw, command-journal, and event-journal hashes; raw artifacts remain separately under `outputs/multiuav/`. The incomplete a28 case has no final raw-trace hash. Available job/pod metadata and console logs preserve scheduler, deadline, and memory failures. These bindings make the new development evidence separable from the frozen accuracy matrices and resource campaign.

Attempt a31 reruns `05e12d8e` in session `e76728cc` under v12 commit `f549a53cfd4f6a5f77cdee2bf96ebf966b3c4b1f`; model, source, seed and command ceiling are unchanged. Its summary and verification record retain all three artifact hashes. The a30 failure remains in Table 3; this is not an independent mission.